CS23334 Fundamentals of Data Science Lab

Experiment 13

Procedure to Handle Missing Values & Inappropriate Data in a Dataset

Objective: To demonstrate data preprocessing techniques for identifying duplicates, removing redundant attributes, handling invalid values, standardizing categorical text, and imputing missing data in a hotel booking dataset (`Hotel_Dataset.csv`).

Methodology:

1. **Duplicate Detection & Removal:** Identify duplicate records using `df.duplicated()` and remove them using `df.drop_duplicates(inplace=True)`, followed by index resetting.
2. **Redundant Attribute Removal:** Drop duplicated or unnecessary columns (e.g. `Age_Group.1`) using `df.drop()`.
3. **Invalid Data Handling:** Convert negative continuous values (Bill, EstimatedSalary) and out-of-bounds ratings/pax values to `np.nan`.
4. **Categorical Standardization:** Clean up inconsistent text representations (`Ibys` → `Ibis`, `Vegetarian/veg` → `Veg`, `non-Veg` → `Non-Veg`).
5. **Statistical Imputation:** Impute missing continuous attributes using feature mean and discrete attributes using feature median.

Code:

```
import numpy as np
import pandas as pd

df = pd.read_csv("Hotel_Dataset.csv")
df

df.duplicated()
df.info()

df.drop_duplicates(inplace=True)
df

len(df)

index = np.array(list(range(0, len(df))))
df.set_index(index, inplace=True)
df

df.drop(['Age_Group.1'], axis=1, inplace=True)
df
```

```
df.CustomerID.loc[df.CustomerID < 0] = np.nan
df.Bill.loc[df.Bill < 0] = np.nan
df.EstimatedSalary.loc[df.EstimatedSalary < 0] = np.nan
df

df['Rating(1-5)'].loc[(df['Rating(1-5)'] < 1) | (df['Rating(1-5)'] > 5)] = np.nan
df['NoOfPax'].loc[(df['NoOfPax'] < 1) | (df['NoOfPax'] > 20)] = np.nan
df

df.Age_Group.unique()
df.Hotel.unique()

df.Hotel.replace(['Ibys'], 'Ibis', inplace=True)

df.FoodPreference.unique()
df.FoodPreference.replace(['Vegetarian', 'veg'], 'Veg', inplace=True)
df.FoodPreference.replace(['non-Veg'], 'Non-Veg', inplace=True)

df.EstimatedSalary.fillna(round(df.EstimatedSalary.mean()), inplace=True)
df.NoOfPax.fillna(round(df.NoOfPax.median()), inplace=True)
df['Rating(1-5)'].fillna(round(df['Rating(1-5)'].median()), inplace=True)
df.Bill.fillna(round(df.Bill.mean()), inplace=True)
df
```

OUTPUT: 1. Initial Dataset, Duplicate Detection & Index Reset

```
In [1]: df = pd.read_csv("Hotel_Dataset.csv")
In [2]: df.duplicated()
Out[2]:
0     False
1     False
2     False
3     False
4     False
5     False
6     False
7     False
8     False
9      True
10    False
dtype: bool

In [3]: df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11 entries, 0 to 10
Data columns (total 9 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   CustomerID       11 non-null     int64
 1   Age_Group        11 non-null     object
 2   Rating(1-5)      11 non-null     int64
 3   Hotel            11 non-null     object
 4   FoodPreference   11 non-null     object
 5   Bill             11 non-null     int64
 6   NoOfPax          11 non-null     int64
 7   EstimatedSalary  11 non-null     int64
 8   Age_Group.1      11 non-null     object

In [4]: df.drop_duplicates(inplace=True)
In [5]: index = np.array(list(range(0, len(df))))
In [6]: df.set_index(index, inplace=True)
In [7]: df.drop(['Age_Group.1'], axis=1, inplace=True)
In [8]: df
Out[8]:
   CustomerID Age_Group  Rating(1-5)       Hotel FoodPreference  Bill  NoOfPax  EstimatedSalary
0           1     20-25            4        Ibis            veg  1300        2            40000
1           2     30-35            5   LemonTree        Non-Veg  2000        3            59000
2           3     25-30            6      RedFox            Veg  1322        2            30000
3           4     20-25           -1   LemonTree            Veg  1234        2           120000
4           5       35+            3        Ibis     Vegetarian   989        2            45000
5           6       35+            3        Ibys        Non-Veg  1909        2           122220
6           7       35+            4      RedFox     Vegetarian  1000       -1            21122
7           8     20-25            7   LemonTree            Veg  2999      -10           345673
8           9     25-30            2        Ibis        Non-Veg  3456        3           -99999
9          10     30-35            5      RedFox        non-Veg  6755        4            87777
```

OUTPUT: 2. Out-of-Bounds Replacement, Categorical Cleaning & Imputed DataFrame

```
In [9]: df.CustomerID.loc[df.CustomerID < 0] = np.nan
In [10]: df.Bill.loc[df.Bill < 0] = np.nan
In [11]: df.EstimatedSalary.loc[df.EstimatedSalary < 0] = np.nan
In [12]: df['Rating(1-5)'].loc[(df['Rating(1-5)'] < 1) | (df['Rating(1-5)'] > 5)] = np.nan
In [13]: df['NoOfPax'].loc[(df['NoOfPax'] < 1) | (df['NoOfPax'] > 20)] = np.nan

In [14]: df.Hotel.replace(['Ibys'], 'Ibis', inplace=True)
In [15]: df.FoodPreference.replace(['Vegetarian', 'veg'], 'Veg', inplace=True)
In [16]: df.FoodPreference.replace(['non-Veg'], 'Non-Veg', inplace=True)

In [17]: df.EstimatedSalary.fillna(round(df.EstimatedSalary.mean()), inplace=True)
In [18]: df.NoOfPax.fillna(round(df.NoOfPax.median()), inplace=True)
In [19]: df['Rating(1-5)'].fillna(round(df['Rating(1-5)'].median()), inplace=True)
In [20]: df.Bill.fillna(round(df.Bill.mean()), inplace=True)
In [21]: df
Out[21]:
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary
0	1.0	20-25	4.0	Ibis	Veg	1300.0	2.0	40000.0
1	2.0	30-35	5.0	LemonTree	Non-Veg	2000.0	3.0	59000.0
2	3.0	25-30	4.0	RedFox	Veg	1322.0	2.0	30000.0
3	4.0	20-25	4.0	LemonTree	Veg	1234.0	2.0	120000.0
4	5.0	35+	3.0	Ibis	Veg	989.0	2.0	45000.0
5	6.0	35+	3.0	Ibis	Non-Veg	1909.0	2.0	122220.0
6	7.0	35+	4.0	RedFox	Veg	1000.0	2.0	21122.0
7	8.0	20-25	4.0	LemonTree	Veg	2999.0	2.0	345673.0
8	9.0	25-30	2.0	Ibis	Non-Veg	3456.0	3.0	96755.0
9	10.0	30-35	5.0	RedFox	Non-Veg	6755.0	4.0	87777.0

Result:

Missing and inappropriate data in the hotel booking dataset were successfully handled. Duplicate records were removed, invalid negative values were replaced with NaN, and missing entries were imputed, resulting in a clean and consistent dataset ready for analysis.